

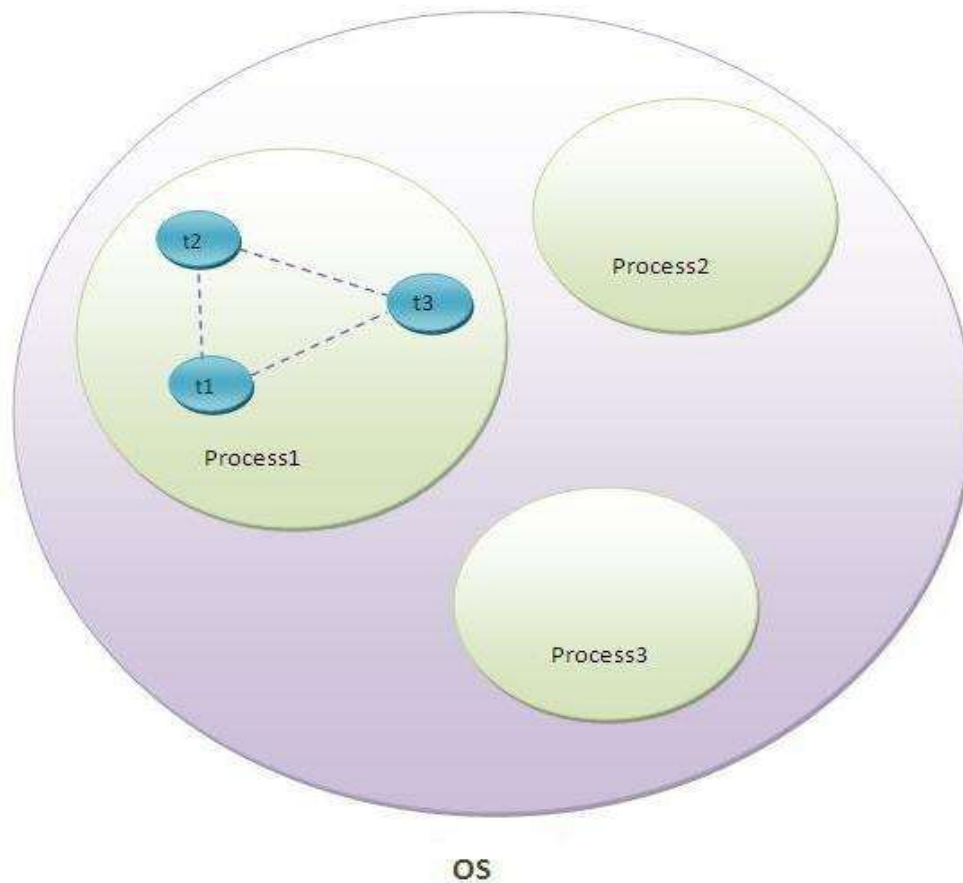
Threads

introduction

- Performing operations concurrently (in parallel)
 - We can walk, talk, breathe, see, hear, smell... all at the same time
 - Computers can do this as well - download a file, print a file, receive email, run the clock, more or less in parallel....
 - How are these tasks typically accomplished?
 - Operating systems support processes
 - What's the difference between a process and a thread?
 - Processes have their own memory space, threads share memory
 - Hence processes are “heavyweight” while threads are “lightweight”
 - Most programming languages do not allow concurrency
 - Java supports concurrency as part of language and libraries

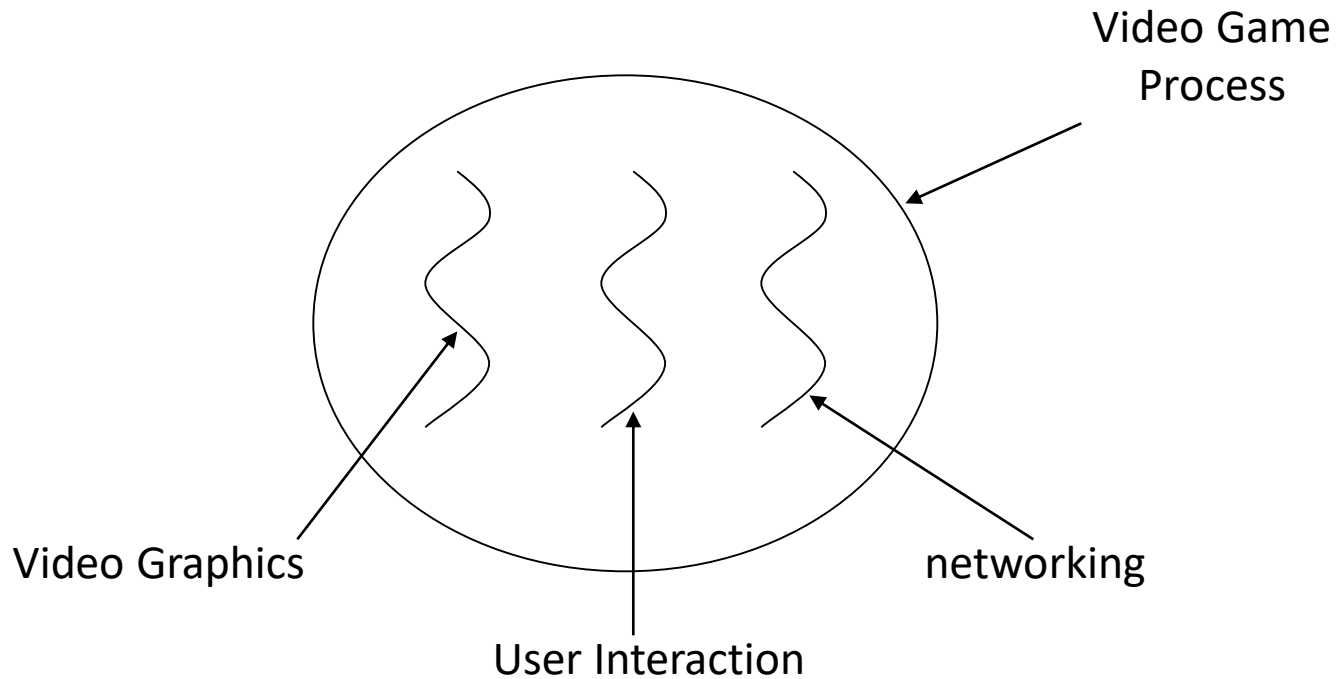
What is a thread?

- A thread is a light weight process.
- Used to provide the ability to perform multiple things at the same time
 - multiple threads can work together to accomplish a common goal
- Video Game example
 - one thread for graphics
 - one thread for user interaction
 - one thread for networking

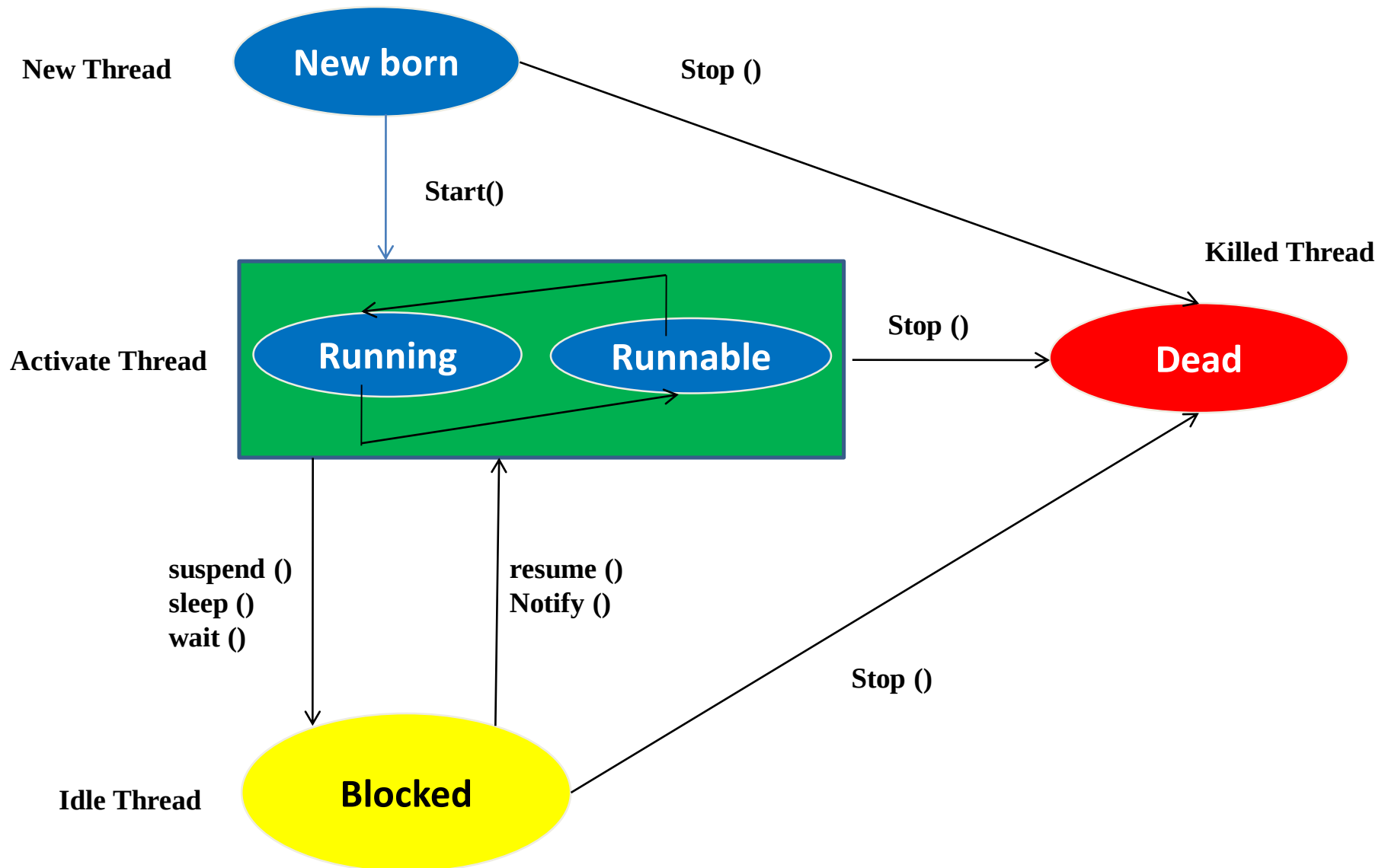


- As shown in the above figure, thread is executed inside the process. There is context-switching between the threads. There can be multiple processes inside the OS and one process can have multiple threads.

What is a Thread? example



Thread States (Life cycle) or thread model



- During the life time of a thread, there are many states it can enter. They are :
 - Newborn state
 - Runnable state
 - Running state
 - Blocked state
 - Dead state
- **Newborn State :**
 - When we create a thread object, the thread is born and is said to be in newborn state. The thread is not yet sheduled for running.

- **Runnable State :**

- Runnable state means that the thread is ready for execution and is waiting for the availability of the processor.
- i.e., the thread has joined the queue of threads that are waiting for execution.

- **Running State :**

- Running means that the processor has given its time to the thread for its execution.
- The thread runs until it relinquishes control on its own.
- A running thread may relinquish its control in one of the following situations:

1) **Suspend() :**

- This method is useful when we want to suspend a thread for some time due to certain reason, but do not want to kill it.
- A suspend thread can be revived by using the **resume()** method.

2) **Sleep() :**

- by this method we can put a thread to sleep for a specified time period using the method **sleep(*time*)** where time is in milliseconds.
- this means that the thread is out of the queue during this time period.
- the thread re-enters the runnable state as soon as this time period is elapsed.

3) **Wait () :**

- it has been told to wait until some event occurs .
- This is done by using the **wait()** method.
- The thread can be scheduled to run again using the **notify()** method

- **Blocked State :**

- A thread is said to be blocked when it is prevented from entering into the runnable state and subsequently the running state.
- This happens when the thread is suspended, sleeping, or waiting in order to satisfy certain requirements.
- A blocked thread is considered “not runnable” but not dead and therefore fully qualified to run again.

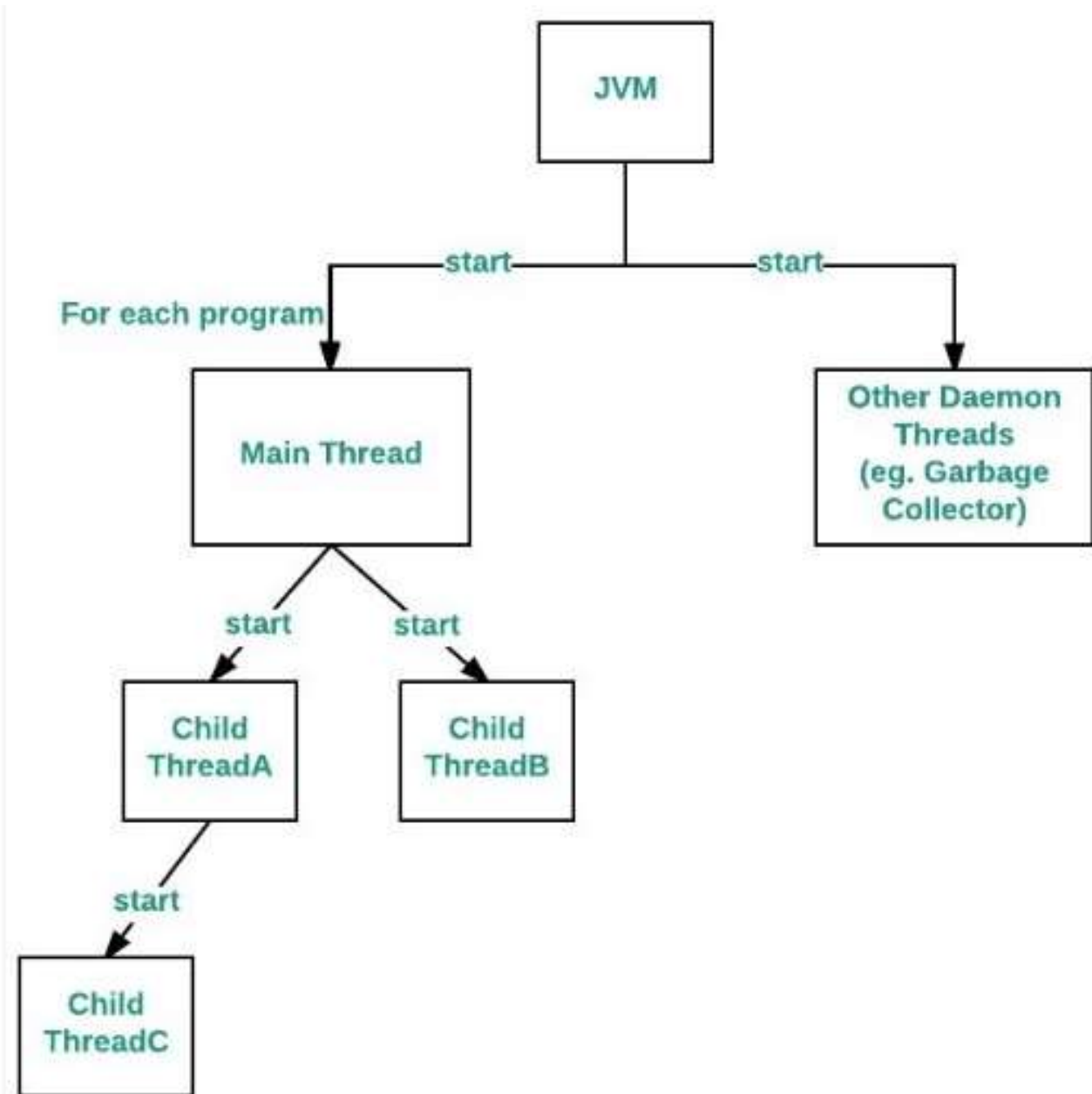
- **Dead State :**

- Every thread has a life cycle. A running thread ends its life when it has completed executing its run() method. It is a natural death.
- However we can kill it by sending the stop message to it at any state thus causing a premature death to it.
- A thread can be killed as soon it is born, or while it is running, or even when it is in blocked state.

The Main thread

- When a Java program starts up, one thread begins running immediately. This is usually called the *main* thread of our program, because it is the one that is executed when our program begins.
- **Properties:**
- It is the thread from which other “child” threads will be spawned.
- Often, it must be the last thread to finish execution because it performs various shutdown actions

- Diagram



- Although the main thread is created automatically when your program is started, it can be controlled through a **Thread** object.
- To do so, you must obtain a reference to it by calling the method **currentThread()**, which is a **public static** member of **Thread**.
- Its general form is shown here:
- `static Thread currentThread( )`

- Eg : Controlling the main Thread.

```
class CurrentThreadDemo
```

```
{
    public static void main(String args[])
    {
        Thread t = Thread.currentThread();
        System.out.println("Current thread: " + t);
        t.setName("My Thread"); // change the name of the thread
        System.out.println("After name change: " + t);
        try
        {
            for(int n = 5; n > 0; n--)
            {
                System.out.println(n);
                Thread.sleep(1000);
            }
        }
        catch (InterruptedException e)
        {
            System.out.println("Main thread interrupted");
        }
    }
}
```

- Output:

Current thread: Thread[main,5,main]

After name change: Thread[My Thread,5,main]

5

4

3

2

1

- Example explanation:
- In this program, a reference to the current thread (the main thread, in this case) is obtained by calling **currentThread()**, and this reference is stored in the local variable **t**.
- Next, the program displays information about the thread.
- The program then calls **setName()** to change the internal name of the thread.
- Information about the thread is then redisplayed. Next, a loop counts down from five, pausing one second between each line.
- The pause is accomplished by the **sleep()** method. The argument to **sleep()** specifies the delay period in milliseconds.
- Notice the **try/catch** block around this loop. The **sleep()** method in **Thread** might throw an **InterruptedException**.

Thread Creation

- You can create a thread by instantiating an object of type Thread.
- There are two ways to create thread in java :
 - Implement the Runnable interface (java.lang.Runnable)
 - By Extending the Thread class (java.lang.Thread)

a) Extending the Thread class :

procedure :

- 1) Declare the class as extending the Thread class.
- 2) Implement the run() method that is responsible for executing the sequence of code that the thread will execute.
- 3) Create a thread object and call the start () method to initiate the thread execution.

1) Declaring a class :

- The thread class can be extended as follows :

```
class MyThread extends Thread  
{  
}
```

now we have a new type of thread MyThread

2) Implementing the run() method

- the run() method has been inherited by the class MyThread.
- The basic implementation of run() will look like this.

```
public void run()  
{  
}
```

3) Starting new thread

To actually create and run an instance of our thread class. We must write the following:

```
MyThread aThread = new MyThread();  
aThread.start();    // invokes run() method
```

- First line indicates that the object is created and instantiated .
i.e., the thread is in newborn state
- Second line calls the start() method causing the thread to move into the runnable state.
- Then, the java runtime will schedule the thread to run by invoking its run() method. Now, the thread is said to be in the running state.

```
class A extends Thread
{
    public void run()
    {
        for(int i=1; i<=5; i++)
        {
            System.out.println("\t From ThreadA : i = "+i);
        }
        System.out.println("Exit from A");
    }
}
class B extends Thread
{
    public void run()
    {
        for(int j=1; j<=5; j++)
        {
            System.out.println("\t From ThreadB : j = "+j);
        }
        System.out.println("Exit from B");
    }
}
```

```
class ThreadTest1
```

```
{
```

```
    public static void main(String args[])
```

```
    {
```

```
        A a = new A();
```

```
        B b = new B();
```

```
        a.start();
```

```
        b.start();
```

```
    }
```

```
}
```

output

From ThreadA : i = 1

From ThreadB : j = 1

From ThreadB : j = 2

From ThreadA : i = 2

From ThreadA : i = 3

From ThreadA : i = 4

From ThreadA : i = 5

From ThreadB : i = 3

From ThreadB : j = 4

Exit from A

From ThreadB : j = 5

Exit from B

Output : (running the same program second time)

From ThreadA : i = 1

From ThreadB : j = 1

From ThreadA : i = 2

From ThreadB : j = 2

From ThreadB : j = 3

From ThreadB : j = 4

From ThreadB : j = 5

From ThreadA : i = 3

Exit from B

From ThreadA : i = 4

From ThreadA : i = 5

Exit from A

Output may vary each time u execute this program.

b) Implement the Runnable interface :

procedure :

1. Declare the class as implementing the Runnable interface.
2. Implement the run() method. An object of this class is a Runnable object.
3. An object of Thread class is created by passing a Runnable object as argument to the Thread constructor. The Thread object now has a Runnable object that implements the run() method.
4. The start() method is invoked on the Thread object created in the previous step.
5. The thread ends when the run() method ends, either by normal completion or by throwing an uncaught exception.

- **Example :**

class **X implements Runnable**

```
{  
    public void run()  
    {  
        for(int i=1; i<=5; i++)  
        {  
            System.out.println("\t ThreadX " +i);  
        }  
        System.out.println("End of ThreadX");  
    } }  
}
```



```
class RunnableTest
```

```
{
```

```
    public static void main(String args[])
```

```
    {
```

```
        X runnable = new X();
```

```
        Thread threadX = new Thread(runnable);
```

```
        threadX.start();
```

```
        System.out.println("End of main Thread");
```

```
    }
```

```
}
```

output

End of the main Thread

ThreadX : 1

ThreadX : 2

ThreadX : 3

ThreadX : 4

ThreadX : 5

End of ThreadX

- Explanation : In main method, we first create an instance of X and then pass this instance as the initial value of the object threadX (an object of thread class). Whenever the new thread threadX starts up, its run() method calls the run() method of the target object supplied to it. Here, the target object is runnable.

Thread priority

- In java, each thread is assigned a priority, which affects the order in which it is scheduled for running.
- Priorities are represented by a number between 1 and 10.
- Java permits us to set the priority of a thread using the `setPriority()` method as follows.
- Syntax:
- `ThreadName.setPriority(intNumber);`
- Thread class defines several priority constants:
- `MIN_PRIORITY = 1`
- `NORM_PRIORITY = 5`
- `MAX_PRIORITY = 10.`
- the default setting is `NORM_PRIORITY`.
- The value of `MIN_PRIORITY` is 1 and the value of `MAX_PRIORITY` is 10

- example:

class **A** extends **Thread**

{

public void run()

{

System.out.println("threadA started");

for(int i=1; i<=4; i++)

{

System.out.println("\tFrom Thread A : i = " +i);

}

System.out.println("Exit From A");

}

}

```
class B extends Thread
```

```
{
```

```
    public void run()
```

```
    {
```

```
        System.out.println("threadB started");
```

```
        for(int j=1; j<=4; j++)
```

```
        {
```

```
            System.out.println("\tFrom Thread B : j = " +j);
```

```
        }
```

```
        System.out.println("Exit From B");
```

```
    }
```

```
}
```

```
class ThreadPriority
```

```
{
```

```
    public static void main(String args[])
```

```
    {
```

```
        A threadA = new A();
```

```
        B threadB = new B();
```

```
        threadB.setPriority(7);
```

```
        threadA.setPriority(3);
```

```
        threadA.start();
```

```
        threadB.start();
```

```
        System.out.println("End of main thread");
```

```
    }
```

```
}
```

- Output:
- End of main thread
- threadA started
- threadB started
- From Thread B : j = 1
- From Thread B : j = 2
- From Thread B : j = 3
- From Thread B : j = 4
- Exit From B
- From Thread A : I = 1
- From Thread A : I = 2
- From Thread A : I = 3
- From Thread A : I = 4
- Exit From A

Thread synchronizations

Synchronization

- When two or more threads need access to a shared resource, they need some way to ensure that the resource will be used by only one thread at a time.
- Java enables us to overcome this problem using a technique called synchronization.
- The Keyword synchronized is used to solve such problems
- This is the general form of the synchronized statement:

```
synchronized(object)
{
    // statements to be synchronized
}
```

- You create a synchronized method by specifying the keyword synchronized in the method's declaration.

```
public synchronized method1()
{
    // statements to be synchronized
}
```

- Only one thread can execute a **synchronized method** at a time
- The `synchronized` keyword in Java creates a block of code referred to as a critical section.
- Every Java object with a critical section of code gets a lock associated with the object.
- To enter a critical section, a thread needs to obtain the corresponding object's lock.
- When ever a thread has completed its work of using `synchronized` method (or a block of code.), it will hand over the key (that can open the lock) to the next thread that is ready to use the same resource

Example

```
class Callme
{
    void call(String msg)
    {
        System.out.print("[Test");
        try
        {
            Thread.sleep(1000);
        }
        catch(InterruptedException e)
        {
            System.out.println("Interrupted");
        }
        System.out.println("]");
    }
}
```

class **Caller** implements **Runnable**

{

Callme target;

 public Caller(Callme **targ**)

 {

target = **targ**;

Thread t=new **Thread(this)**;

t.start();

 }

 public void run()

 {

target.call();

 }

}

```
class Synch
{
    public static void main(String args[])
    {
        Callme target = new Callme();
        Caller ob1 = new Caller(target);
        Caller ob2 = new Caller(target);
        Caller ob3 = new Caller(target);
    }
}
```

output

```
[ Hello [ Synchronized [ World ]
]
]
```

Save as Synch.java

- As you can see, by calling `sleep()`, the `call()` method allows execution to switch to another thread. This results in the mixed-up output of the three message strings.
- In this program, nothing exists to stop all three threads from calling the same method, on the same object, at the same time. This is known as **race condition**.
- Becoz the three threads are racing each other to complete the method. This example used `sleep()` to make the effects repeatable and obvious.
- To fix the preceding program, you must serialize access to `call()`.
- i.e., you must restrict its access to only one thread at a time. To do this we simply need to precede `call()`'s definition with the keyword **synchronized**.
- Therefore in our program

```
class Callme()  
{  
    synchronized void call(String msg){ }  
}
```

- This prevents other threads from entering call() while another thread is using it.
- After synchronized has been added to call(), the output of the program is as follows.

[Hello]

[Synchronized]

[World]

Multithreading

- **Multitasking** : An operating system can execute several programs simultaneously. This ability is called multitasking.
- In system's terminology , it is called multithreading.
- **Multithreading** : In multithreading a program(process) is divided into two or more subprograms (processes), which can be implemented at the same time in parallel

• Multithreading

It is a programming concept in which a program(process) is divided into two or more subprograms or threads that are executed at the same time in parallel.

It supports execution of multiple parts of a single program simultaneously.

A thread is the smallest unit in multithreading

It helps in developing efficient programs

Multitasking

It is an operating system concept in which multiple tasks are performed simultaneously.

It supports execution of multiple programs simultaneously

A program or process is the smallest unit in a multitasking environment.

It helps in developing efficient operating systems.

```
class A extends Thread    // same old example for thread//
{
    public void run()
    {
        for(int i=1; i<=5; i++)
        {
            System.out.println("\t From ThreadA : i = "+i);
        }
        System.out.println("Exit from A");
    }
}

class B extends Thread
{
    public void run()
    {
        for(int j=1; j<=5; j++)
        {
            System.out.println("\t From ThreadB : j = "+j);
        }
        System.out.println("Exit from B");
    }
}
```

```
class ThreadTest1
```

```
{
```

```
    public static void main(String args[])
```

```
    {
```

```
        A a = new A();
```

```
        B b = new B();
```

```
        a.start();
```

```
        b.start();
```

```
    }
```

```
}
```

output

From ThreadA : i = 1

From ThreadB : j = 1

From ThreadB : j = 2

From ThreadA : i = 2

From ThreadA : i = 3

From ThreadA : i = 4

From ThreadA : i = 5

From ThreadB : i = 3

From ThreadB : j = 4

Exit from A

From ThreadB : j = 5

Exit from B

Output : (running the same program second time)

From ThreadA : i = 1

From ThreadB : j = 1

From ThreadA : i = 2

From ThreadB : j = 2

From ThreadB : j = 3

From ThreadB : j = 4

From ThreadB : j = 5

From ThreadA : i = 3

Exit from B

From ThreadA : i = 4

From ThreadA : i = 5

Exit from A

Output may vary each time u execute this program.

Inter thread communication

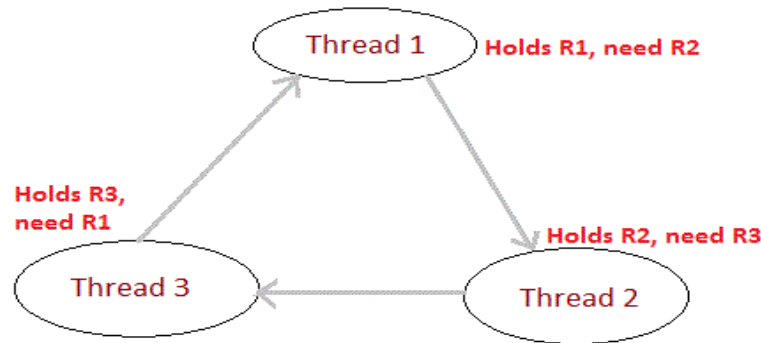
- **Inter-thread communication** is all about allowing synchronized threads to communicate with each other. i.e.,
- Inter-thread communication can be defined as the exchange of messages between two or more threads.
- Java implements inter-thread communication with the help of following three methods:
- 1. notify():
 - Resumes the first thread that went into the sleep mode. Eg:
 - final void notify()
- 2. notifyall()
 - Resumes all the threads that are in sleep mode. Eg:
 - final void notifyall()
- 3. wait()
 - Sends the calling thread into the sleep mode. This thread can now be activated only by notify() or notifyall() methods.
 - final void wait()

- Java provides complete control over multithreaded program.
- You can develop a multithreaded program which can be suspended, resumed, or stopped completely based on your requirements.
- There are various static methods which you can use on thread objects to control their behavior.
- **suspend()**
- This method puts a thread in the suspended state and can be resumed using resume() method.
- **stop()**
- This method stops a thread completely.
- **resume()**
- This method resumes a thread, which was suspended using suspend() method.

Deadlock

- Deadlock describes condition where two or more threads are blocked forever, waiting for each other.

Thread Deadlock in Java



Deadlock Condition

- Deadlock is a situation of complete Lock, when no thread can complete its execution because lack of resources. In the above picture, Thread 1 is holding a resource R1, and need another resource R2 to finish execution, but R2 is locked by Thread 2, which needs R3, which in turn is locked by Thread 3. Hence none of them can finish and are stuck in a deadlock.

- Thread deadlock example:

```
public class DeadlockDemo
{
    String s1 = "Window";
    String s2 = "Linux";
    Thread t1 = new Thread(" Thread 1")
    {
        public void run()
        {
            while(true)
            {
                synchronized(s1)
                {
                    synchronized(s2)
                    {
                        System.out.println(s1 + s2);
                    }
                }
            }
        }
    };
};
```

```
Thread t2 = new Thread(" Thread 2")
{
    public void run()
    {
        while(true)
        {
            synchronized(s2)
            {
                synchronized(s1)
                {
                    System.out.println(s2 + s1);
                }
            }
        }
    }
};
public static void main(String a[])
{
    DeadlockDemo dd = new DeadlockDemo();
    dd.t1.start();
    dd.t2.start();
}
}
```